

THE 6 MEMORY CONTEXT TYPES

decision — Why we chose X over Y

Not a rule (rules enforce; decisions explain why)

Examples: `adr-001-database-choice` PostgreSQL over MongoDB • `auth-strategy` Auth0 vs custom

architecture — What the system looks like

Not a skill (skills do things; architecture describes things)

Examples: `system-topology` services, DBs, queues • `api-contracts` endpoints & schemas

incident — What happened and root cause

Not a runbook (runbooks are skills; incidents are evidence)

Examples: `incident-2026-03-redis` failover postmortem • `deploy-rollback-apr`

domain — What things mean in our context

Not a constraint (rules constrain; domain knowledge informs)

Examples: `domain-payments` EU regulations • `glossary` tenant, deploy ≠ release

context — Who/when/where around the project

Not an agent (agents act; context describes the environment)

Examples: `team-ownership` who owns what • `q2-priorities` focus areas

learning — What we measured and observed

Not a policy (rules prescribe; learnings provide evidence)

Examples: `caching-strategy` p99 120ms→8ms • `reserved-instances` saved 40%

MEMORY FRONTMATTER

```
---
name: api-versioning-strategy
description: Team decision on URL-based API versioning
version: 1.0.0
author: alice
project: platform
scope: global
context_type: decision
related: [code-reviewer, api-gateway]
# Body: markdown content below
```

KEY BOUNDARY

Memories store **knowledge** — things an agent needs to **know**.
They don't store: actions (→ skill), constraints (→ rule), instructions (→ prompt), actor definitions (→ agent).

WHEN TO USE EACH TYPE

context_type	Use when...
decision	A team chose X over Y and you want to capture why
architecture	You need to document how the system is structured
incident	Something went wrong and you want to preserve the postmortem
domain	Business rules or concepts need to be consistent across the team
context	Team structure, priorities, or project state needs sharing
learning	You learned something valuable from experience

ARTIFACT TYPE BOUNDARIES

Type	Stores	Does NOT store
Agent	Actor definitions, orchestration	Knowledge (→ memory), constraints (→ rule)
Skill	Procedures, automation, how-to	Why we do X (→ memory), what X must follow (→ rule)
Rule	Constraints, policies, standards	Why it was decided (→ memory), how to implement (→ skill)
Memory	Knowledge, context, evidence	Actions (→ skill), constraints (→ rule), prompts
Prompt	Instruction templates for AI	Execution logic (→ skill), actor definition (→ agent)

KNOWLEDGE MAPPING FOR IT PROJECTS

Maps every situation where valuable knowledge is generated to the appropriate memory context type and the roles involved.

Requirements & Analysis

Situation	Valuable Knowledge	Type	Roles
Stakeholder interviews	Business needs, personas, pain points	domain	PO, BA → Dev
Functional requirements	What the system must do, acceptance criteria	domain	BA → Dev, QA
Non-functional reqs	SLAs, latency targets, availability	domain	Arch, SRE → Dev
Regulatory constraints	GDPR, SOC2, HIPAA, local laws	domain	Legal → All
Glossary	"Tenant means X. Deploy ≠ Release"	domain	BA, TL → All
User journeys	System flows, drop-off points	domain	UX → Frontend

Architecture & Design

Situation	Valuable Knowledge	Type	Roles
System topology	Services, DBs, queues, connections	architecture	Arch → All
Data models / ERDs	Entities, relationships, indexes	architecture	Arch, DBA → Dev
API contracts	Endpoints, schemas, versioning	architecture	Backend → Frontend
Network topology	VPCs, subnets, firewall rules	architecture	Cloud → SRE
CI/CD pipeline	Stages, environments, rollback	architecture	DevOps → Dev
Security architecture	Auth flows, encryption, key mgmt	architecture	SecArch → All
DR design	RPO/RTO, backup, failover regions	architecture	Arch, SRE → Ops

Decisions

Situation	Valuable Knowledge	Type	Roles
Technology selection	"PostgreSQL over MongoDB because..."	decision	Arch, TL → All
Framework choice	"Next.js over Remix because..."	decision	TL → Dev
Build vs buy	"Auth0 instead of custom auth because..."	decision	CTO → All
Cloud provider	"AWS over Azure because..." + cost analysis	decision	CTO, Cloud → All
Migration strategy	Strangler fig vs big bang, why	decision	Arch, PM → Dev
Vendor selection	"Datadog over Grafana Cloud because..."	decision	Platform → SRE
Trade-off records	"Accepted eventual consistency because..."	decision	Arch → Dev, QA

Incidents & Postmortems

Situation	Valuable Knowledge	Type	Roles
Production outage	Timeline, root cause, blast radius	incident	SRE → All
Security breach	Attack vector, containment, disclosure	incident	Sec, SRE → Mgmt
Data loss event	Recovery steps, data integrity status	incident	DBA, SRE → Mgmt
Failed migration	What broke, rollback, data state after	incident	DBA → Dev, PM
Failed deployment	Why canary/checks didn't catch it	incident	DevOps → SRE
Capacity incident	Traffic spike, auto-scaling failure	incident	SRE → FinOps
Third-party outage	Vendor impact, fallback, SLA claim	incident	SRE → Mgmt
Near miss	"Caught in staging because..."	incident	Any → All

Team & Project Context

Situation	Valuable Knowledge	Type	Roles
Team ownership	Who owns which service, escalation	context	EM → All
Quarterly priorities	Focus areas, what's deprioritized	context	PM → Dev
Stakeholder map	Who cares about what, RACI	context	PM, BA → All
Project timeline	Milestones, deadlines, blockers	context	PM → All
Budget constraints	Cloud budget, headcount, licensing	context	FinOps → Arch
Vendor relationships	Contract terms, renewal dates	context	Procurement → SRE
Compliance deadlines	Audit dates, certification renewals	context	Legal → All
Onboarding notes	"Things I wish I'd known day 1"	context	New hires → Future

Learnings & Evidence

Situation	Valuable Knowledge	Type	Roles
Perf benchmarks	"Caching: p99 120ms → 8ms"	learning	SRE → Dev
Cost optimization	"Reserved instances saved 40%"	learning	FinOps → Mgmt
Testing strategy	Which tests catch real bugs vs noise	learning	QA → Dev
Migration retro	What worked, time estimates vs actuals	learning	TL, PM → Future
Tool evaluation	"Tried X for 2 months, here's findings"	learning	Any → All
Scaling findings	"10K rps before degrading, bottleneck is X"	learning	SRE → Arch
Security audit	Pentest results, remediation	learning	Sec → Dev, SRE
Failed experiment	"Tried X, didn't work because Y"	learning	Any → All
DR drill results	Recovery time actual vs target, gaps	learning	SRE → Mgmt